

Proyecto Final de Curso: Plataforma Web Resiliente con Kubernetes

Autor: Hugo González De Eiris Pineda

1. Introducción

Este documento es la memoria de mi proyecto final del ciclo ASIR. Mi objetivo es desplegar una plataforma web completa y resiliente, basada en **WordPress** y **MySQL**.

Para ello, mi enfoque se basa en tecnologías modernas de contenedores: he decidido encapsular cada servicio (la web, la base de datos, etc.) en su propio contenedor **Docker**. La gestión, escalabilidad y disponibilidad de toda esta infraestructura la realizaré mediante **Kubernetes**.

2. Aspectos Clave del Proyecto

Centro mi proyecto en los siguientes aspectos clave para construir una solución robusta y profesional:

- **Alta Disponibilidad y Resiliencia:** He diseñado la arquitectura para que el servicio continúe funcionando incluso si fallan componentes. Lo conseguiré mediante réplicas de contenedores y balanceo de carga.
- **Persistencia de Datos:** Es fundamental que los datos no se pierdan. Utilizaré volúmenes persistentes en Kubernetes para que la base de datos sobreviva aunque el contenedor de MySQL se reinicie o falle.
- **Seguridad:** Implementaré HTTPS para cifrar toda la comunicación con el sitio web, asegurando la integridad de los datos.
- **Monitorización y Gestión:** No basta con que funcione, hay que saber *cómo* funciona. Integraré **Prometheus** y **Grafana** para monitorizar el estado del clúster y **phpMyAdmin** para gestionar la base de datos.
- **Aprendizaje Profundo:** Un reto personal que he asumido es la **creación manual de las imágenes Docker** para MySQL y WordPress. En lugar de usar las imágenes oficiales, crearé las mías desde cero para entender a fondo su funcionamiento interno.

3. Stack Tecnológico

Para llevar a cabo este proyecto, utilizaré el siguiente conjunto de tecnologías:

- **Lenguajes de Configuración:** SQL, YAML, Dockerfile
- **Contenerización:** Docker
- **Orquestación:** Kubernetes (K3s)
- **Aplicación Web:** WordPress
- **Base de Datos:** MySQL/MariaDB
- **Servidor Web/Proxy Inverso:** Nginx
- **Monitorización:** Prometheus, Grafana
- **Administración de Base de Datos:** phpMyAdmin

4. Bitácora de Realización del Proyecto

En esta sección, documento paso a paso mi trabajo práctico, los comandos que utilizo y los problemas que voy solucionando.

Fase 1: Preparación del Entorno (VM, Docker y K3s)

Mi descripción:

En esta fase inicial, preparo la máquina virtual (Ubuntu Server 22.04 con 50GB de disco) e instalo las herramientas base: Docker y K3s.

Comandos y Pasos:

Mi primer paso: 1. Actualizo los paquetes

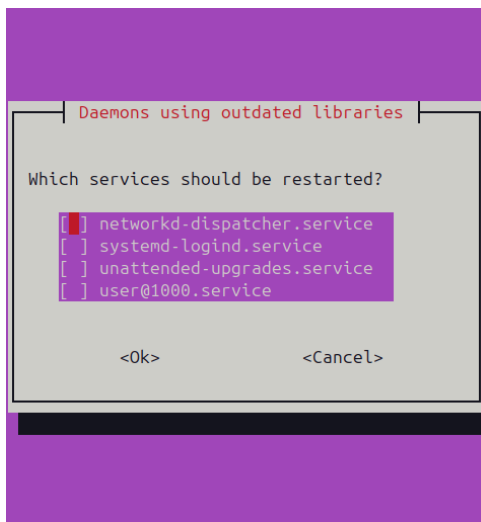
Primero, me aseguro de que mi lista de paquetes esté actualizada para evitar problemas de dependencias.

sudo apt update && sudo apt upgrade -y

Mi siguiente paso: 2. Instalo las dependencias

Instalo los paquetes necesarios para que apt pueda usar mis repositorios a través de HTTPS.

sudo apt install -y apt-transport-https ca-certificates curl software-properties-common gnupg lsb-release



Le doy a Ok sin seleccionar nada

Mi siguiente paso: 3. Añado la clave GPG de Docker

Descargo y añado la clave de seguridad oficial de Docker para verificar la autenticidad de mis paquetes.

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

Mi siguiente paso: 4. Configuro el repositorio de Docker

Añado el repositorio oficial de Docker a la lista de fuentes de software de mi sistema.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Mi siguiente paso: 5. Instalo Docker Engine

Actualizo mi lista de paquetes (ahora incluyendo el repo de Docker) e instalo Docker y sus componentes.

```
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

Mi siguiente paso: 6. Configuro mis permisos de usuario

Añado mi usuario actual (\$USER) al grupo docker. Esto me permite ejecutar comandos de Docker sin necesidad de sudo

```
sudo usermod -aG docker $USER
```

Ahora reinicio mi máquina.

Y verifico que todo está perfecto:

```
usuario@usuario:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:56433a6be3fda188089fb548eae3d91df3ed0d6589f7c2656121b911198df065
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Descargo y ejecuto una imagen de prueba **sin sudo** para confirmar que Docker me funciona y que mis permisos de usuario se aplicaron correctamente.

Mi siguiente paso: 7. Instalo Kubernetes (K3s)

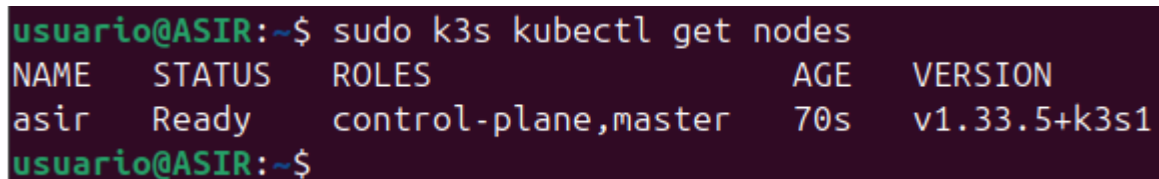
Descripción de mi comando: Como mi intento anterior falló debido al error de compatibilidad (pipefail), ejecuto el script de instalación de K3s forzando la interpretación del script con /bin/bash. Esto me instala K3s correctamente y me asegura que mi archivo kubeconfig sea accesible.

```
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="--write-kubeconfig-mode 644" /bin/bash
```

Mi siguiente paso: 8. Verifico mi instalación de Kubernetes

Compruebo el estado de los nodos de mi clúster K3s para confirmar que mi nodo maestro está operativo y en estado Ready

```
sudo k3s kubectl get nodes
```



```
usuario@ASIR:~$ sudo k3s kubectl get nodes
NAME      STATUS    ROLES                  AGE     VERSION
asir      Ready     control-plane,master   70s     v1.33.5+k3s1
usuario@ASIR:~$
```

FASE 2: IMPLEMENTACIÓN - CREACIÓN DE IMÁGENES DOCKER PERSONALIZADAS

Mi siguiente paso: 9. Creo mi estructura del proyecto para MySQL

Creo mi directorio principal del proyecto (k8s-wordpress-project) en mi directorio personal (~) y mi subcarpeta (mysql-image) para mi Dockerfile y mis archivos de configuración de la base de datos.

```
cd ~
mkdir k8s-wordpress-project
cd k8s-wordpress-project
mkdir mysql-image
```

Mi siguiente paso: 10. Creo mi Dockerfile para MySQL

Creo mi archivo Dockerfile para mi imagen personalizada de MySQL, usando el editor nano. Este Dockerfile define mi imagen base y mis dependencias de configuración.

nano mysql-image/Dockerfil

```
# Uso mi imagen oficial base de MariaDB (compatible con MySQL)
FROM mariadb:10.11
```

```
# Defino mi versión y mi autor (¡IMPORTANTE para mi documentación!)
LABEL maintainer="Hugo Gonzalez De Eiris Pineda"
LABEL version="1.0-custom"
```

```
# Creo mi directorio donde pondré mi archivo de configuración personalizado.
RUN mkdir -p /etc/mysql/conf.d
```

```
# Copio mi archivo de configuración MySQL personalizado
COPY custom.cnf /etc/mysql/conf.d/custom.cnf
```

```
# Copio mi script SQL de inicialización (se me ejecuta automáticamente al arrancar).
COPY init_db.sql /docker-entrypoint-initdb.d/
```

Mi siguiente paso: 11. Creo mi archivo de configuración (custom.cnf)

Creo mi archivo de configuración de MySQL. Este archivo es crucial para mí porque fuerza a mi base de datos a usar el charset correcto (utf8mb4), tal como le indico en mi Dockerfile. Esto me evita problemas con WordPress y los caracteres especiales.

nano mysql-image/custom.cnf

```
[mysqld]
# Forzar el charset a UTF8 para evitar problemas de compatibilidad con WordPress
character-set-server=utf8mb4
collation-server=utf8mb4_unicode_ci

# Mi ajuste opcional de buffers
innodb_buffer_pool_size=128M
usuario@usuario:~/k8s-wordpress-project/mysql-image$
```

Mi siguiente paso: 12. Creo mi script de inicialización (init_db.sql)

Creo mi script SQL que se me ejecutará la primera vez que se inicie mi contenedor. Este script crea mi base de datos wordpress_db de WordPress, asegurando que mi motor se configure al arrancar.

nano mysql-image/init_db.sql

```
CREATE DATABASE IF NOT EXISTS wordpress_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

Mi siguiente paso: 13. Construyo mi imagen Docker de MySQL

Construyo mi imagen personalizada de MySQL a partir de mi Dockerfile y mis archivos de configuración que he creado. Le asigno mi nombre (mysql-personalizado) y mi etiqueta (v1.0).

docker build -t mysql-personalizado:v1.0 .

```
usuario@usuario:~/k8s-wordpress-project/mysql-image$ docker build -t mysql-personalizado:v1.0 .
[+] Building 0.6s (9/9) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 631B                             0.0s
=> [internal] load metadata for docker.io/library/mariadb:10.11 0.4s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/4] FROM docker.io/library/mariadb:10.11@sha256:28c679249b2ede0faf42ba7fbc13b538f 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 62B                                   0.0s
=> CACHED [2/4] RUN mkdir -p /etc/mysql/conf.d                  0.0s
=> CACHED [3/4] COPY custom.cnf /etc/mysql/conf.d/custom.cnf   0.0s
=> CACHED [4/4] COPY init_db.sql /docker-entrypoint-initdb.d/  0.0s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:1b2393775edae07c3a2b7061007589d421917576cd6a6640da5ea9f933a 0.0s
=> => naming to docker.io/library/mysql-personalizado:v1.0     0.0s
```

Mi siguiente paso: 14. Creo mi estructura del proyecto para WordPress

Creo mi subcarpeta (wordpress-image) dentro de mi directorio principal del proyecto (~/.k8s-wordpress-project) para empezar a construir mi imagen de WordPress.

cd ~/.k8s-wordpress-project

mkdir wordpress-image

cd wordpress-image

Mi siguiente paso: 15. Creo mi Dockerfile para WordPress

Creo mi Dockerfile para mi imagen de WordPress. Esta imagen debe incluir mis extensiones PHP y mi propia etiqueta de autoría. Uso mi imagen base de PHP (php:8.2-fpm-alpine) porque me es ligera y luego le añado WordPress.

nano Dockerfile

```
# Usamos una imagen base oficial de PHP con Alpine (muy ligera)
FROM php:8.2-fpm-alpine

# Definimos el autor y versión (para la documentación)
LABEL maintainer="Hugo Gonzalez De Eiris Pineda"
LABEL version="1.0-custom"

# 1. Instalamos dependencias de sistema y extensiones PHP necesarias p
RUN apk update && apk add --no-cache \
    mysql-client \
    git \
    libzip-dev \
    libpng-dev \
    libjpeg-turbo-dev \
    freetype-dev \
    icu-dev \
    zlib-dev \
    libxml2-dev \
    && docker-php-ext-install mysqli opcache exif pdo pdo_mysql \
    && docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install gd \
    && rm -rf /var/cache/apk/*

# 2. Descargamos la última versión de WordPress y la descomprimos.
RUN curl -o /tmp/wordpress.tar.gz -fSL https://wordpress.org/latest.ta
    && tar -xzf /tmp/wordpress.tar.gz -C /usr/src/ \
    && rm /tmp/wordpress.tar.gz

# 3. Copiamos los archivos de WordPress al directorio de trabajo
RUN cp -r /usr/src/wordpress/. /var/www/html/

# 4. Aseguramos que el usuario "www-data" sea dueño del directorio web
RUN chown -R www-data:www-data /var/www/html

# 5. Expongo el puerto 9000 (el puerto por defecto de PHP-FPM)
EXPOSE 9000

# 6. Ejecuto el comando de inicio por defecto de PHP-FPM
CMD ["php-fpm"]
```

Mi siguiente paso: 16. Construyo mi imagen Docker de WordPress

Construyo mi imagen personalizada de WordPress a partir de mi Dockerfile que acabo de crear. Le asigno mi nombre (wordpress-personalizado) y mi etiqueta (v1.0).

docker build -t wordpress-personalizado:v1.0 .

```
usuario@usuario:~/k8s-wordpress-project/wordpress-image$ docker build
do:v1.0 .
[+] Building 149.1s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.35kB
=> [internal] load metadata for docker.io/library/php:8.2-fpm-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/5] FROM docker.io/library/php:8.2-fpm-alpine@sha256:f5cb
=> [2/5] RUN apk update && apk add --no-cache      mysql-client      gi
=> [3/5] RUN curl -o /tmp/wordpress.tar.gz -fSL https://wordpress.org
=> [4/5] RUN cp -r /usr/src/wordpress/. /var/www/html/
=> [5/5] RUN chown -R www-data:www-data /var/www/html
=> exporting to image
=> => exporting layers
=> => writing image sha256:e90ed7b5dcdbc8e3b970342fa9561601a450669783
=> => naming to docker.io/library/wordpress-personalizado:v1.0
```

Mi siguiente paso: 17. Creo mi manifiesto de Persistencia (PV y PVC)

Creo mi archivo mysql-persistence.yaml. Este me define mi **PersistentVolume** (PV) y mi **PersistentVolumeClaim** (PVC). Esto me es crucial para asegurar que mi base de datos de MySQL no pierda mis datos al reiniciarse (así cumplo mi requisito de persistencia).

nano mysql-persistence.yaml

```
usuario@usuario: ~/k8s-wordpress-project
GNU nano 6.2                                mysql-persistence.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/mysql-data"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

Mi siguiente paso: 18. Creo mi directorio HostPath

Mi PV que acabo de definir requiere que mi directorio físico /mnt/data/mysql-data exista en mi máquina virtual para poder almacenar mis datos de MySQL.

sudo mkdir -p /mnt/data/mysql-data

```
usuario@usuario: ~/k8s-wordpress-project
usuario@usuario:~/k8s-wordpress-project$ sudo mkdir -p /mnt/data/mysql-data
[sudo] password for usuario:
usuario@usuario:~/k8s-wordpress-project$
```

Mi siguiente paso: 19. Aplico mi manifiesto de Persistencia

Uso kubectl para aplicar mi manifiesto, creando mis recursos de PV y PVC en mi clúster K3s.

sudo k3s kubectl apply -f mysql-persistence.yaml

```
usuario@usuario: ~/k8s-wordpress-project
usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl apply -f mysql-persistence.yaml
persistentvolume/mysql-pv created
persistentvolumeclaim/mysql-pvc created
usuario@usuario:~/k8s-wordpress-project$
```

FASE 3: ORQUESTACIÓN CON KUBERNETES

Despliego mi servicio de MySQL y mi **Secret** (mis credenciales seguras).

Mi siguiente paso: 20. Creo mi Secret de Credenciales

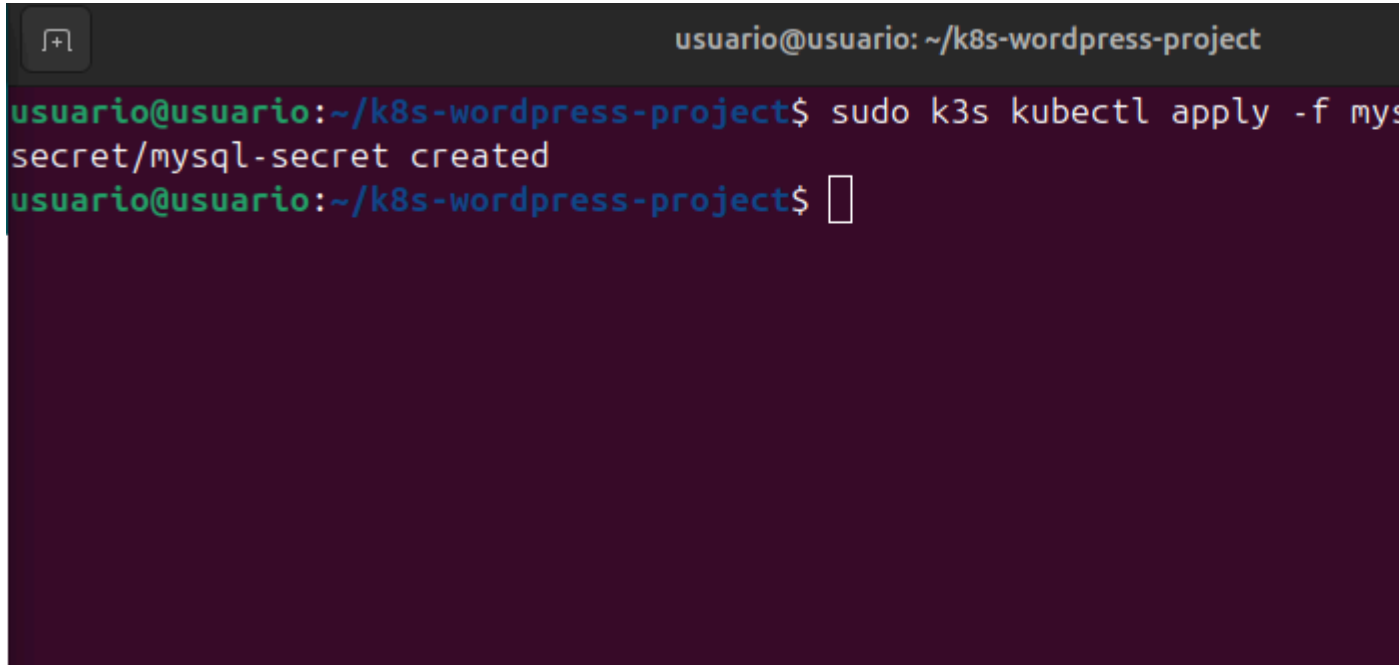
Creo mi Secret de Kubernetes. Esto me es fundamental para mi seguridad, ya que almaceno mis contraseñas de MySQL codificadas en Base64 en lugar de dejarlas en texto plano en mis archivos YAML.

```
usuario@usuario: ~/k8s-wordpress-project
GNU nano 6.2 mysql-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
type: Opaque
data:
  MYSQL_DATABASE: d29yZHB5ZXNzX2Ri
  MYSQL_USER: d3BfdXNlcg==
  MYSQL_PASSWORD: cDRzc3cwcmRTM2d1cm8=
  MYSQL_ROOT_PASSWORD: cm9vdFMzZ3Vybw==
```

Mi siguiente paso: 21. Aplico mi Secret

Aplico mi manifiesto del Secret para que Kubernetes pueda usar mis credenciales seguras.

```
sudo k3s kubectl apply -f mysql-secret.yaml
```



```
usuario@usuario: ~/k8s-wordpress-project
usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl apply -f mysql-secret.yaml
secret/mysql-secret created
usuario@usuario:~/k8s-wordpress-project$
```

Mi paso: 22. Creo mi manifiesto de MySQL (Deployment + Service)

Creo mi archivo mysql-deployment.yaml. Este archivo me define cómo despliegó mi base de datos:

1. **Mi Deployment:** Uso mi imagen personalizada (mysql-personalizado:v1.0), mis credenciales del Secret y monto mi PVC para mi persistencia.
2. **Mi Service:** Creo mi nombre DNS interno (mysql-service) para que mi WordPress pueda encontrar mi BBDD.

nano mysql-deployment.yaml

```
usuario@usuario: ~/k8s-wordpress-project
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    app: wordpress-mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress-mysql
  template:
    metadata:
      labels:
        app: wordpress-mysql
    spec:
      containers:
        - name: mysql
          image: mysql-personalizado:v1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 3306
          envFrom:
            - secretRef:
                name:
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
  labels:
    app: wordpress-mysql
spec:
  ports:
    - port: 3306
  selector:
    app: wordpress-mysql
  clusterIP: None
```

Mi paso: 23. Aplico mi manifiesto de MySQL

Despliego mi base de datos MySQL en mi clúster. Esto me creará mi Deployment (mi Pod) y mi Service (mi nombre DNS interno).

```
sudo k3s kubectl apply -f mysql-deployment.yaml
```

Mi siguiente paso: 24. Elimino mi Deployment fallido

Uso kubectl delete para eliminar mi Deployment que me falló en mi primer intento. Esto me limpia mi estado anterior y me permite aplicar mi configuración corregida sin conflictos.

```
sudo k3s kubectl delete deployment mysql-deployment
```

Mi paso: 25. Re-creo mi manifiesto de MySQL (Deployment + Service)

Recreo mi archivo mysql-deployment.yaml para asegurarme de que mi configuración sea correcta y apunte a mi Secret que ya existe. Este manifiesto me define mi Pod de la base de datos y mi Servicio para que mi WordPress pueda encontrarlo.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    app: wordpress-mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress-mysql
  template:
    metadata:
      labels:
        app: wordpress-mysql
    spec:
      containers:
        - name: mysql
          image: mysql-personalizado:v1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 3306
          envFrom:
            - secretRef:
                name: mysql-secret
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
  labels:
    app: wordpress-mysql
spec:
  ports:
    - port: 3306
  selector:
    app: wordpress-mysql
EOF
clusterIP: None
```

Mi paso: 26. Aplico mi manifiesto de MySQL (Definitivo)

Despliego mi base de datos MySQL en mi clúster. Ahora que mi Secret existe y mi archivo está limpio, esto me crea mi Pod y mi Servicio correctamente.

```
usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl apply -f mysql-deployment.apps/mysql-deployment created
Warning: resource services/mysql-service is missing the kubectl.kubernetes.io/applyconfiguration annotation which is required by kubectl apply. kubectl apply will only create resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
service/mysql-service configured
usuario@usuario:~/k8s-wordpress-project$
```

FASE 4: DESPLIEGUE DE WORDPRESS (Arquitectura Sidecar)

Mi siguiente paso: 30. Creo mi Persistencia para WordPress

Creo mi directorio físico en mi VM y mi manifiesto wordpress-persistence.yaml para asegurar que mis archivos de la web (plugins, temas, etc.) no se me pierdan.

1. Creo mi directorio físico

```
sudo mkdir -p /mnt/data/wordpress-data
```

2. Creo mi manifiesto PV y PVC

```
sudo tee wordpress-persistence.yaml <<EOF
```

```
apiVersion: v1
```

```
kind: PersistentVolume
```

```
metadata:
```

```
  name: wordpress-pv
```

```
spec:
```

```
  capacity:
```

```
    storage: 5Gi
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
  hostPath:
```

```
    path: "/mnt/data/wordpress-data"
```

```
---
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: wordpress-pvc
```

```
spec:
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
  resources:
```

```
    requests:
```

```
      storage: 5Gi
```

```
EOF
```

3. Una vez que lo haya pegado, ejecuto mi comando para aplicar:

sudo k3s kubectl apply -f wordpress-persistence.yaml

```
usuario@usuario:~$ sudo tee wordpress-persistence.yaml <<EOF
apiVersion: v1
kind: PersistentVolume
metadata:
  name: wordpress-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/wordpress-data"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: wordpress-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
EOF
apiVersion: v1
kind: PersistentVolume
metadata:
  name: wordpress-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/wordpress-data"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: wordpress-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
usuario@usuario:~$ sudo k3s kubectl apply -f wordpress-persistence.yaml
persistentvolume/wordpress-pv unchanged
persistentvolumeclaim/wordpress-pvc created
usuario@usuario:~$ ^C
usuario@usuario:~$
```

Mi paso 31: Configuro mi "Cerebro" de Nginx (ConfigMap)

Nginx no sabe por sí solo cómo hablar con mi WordPress. Necesito darle un archivo de configuración que le diga: "Si alguien pide un archivo .php, dáselo a mi contenedor de WordPress en el puerto 9000".

```
sudo tee wordpress-nginx-config.yaml <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  default.conf: |
    server {
      listen 80;
      root /var/www/html;
      index index.php index.html;

      location / {
        try_files $uri $uri/ /index.php?$args;
      }

      location ~ \.php$ {
        fastcgi_split_path_info ^(.+\.php)(/.+)$;
        fastcgi_pass 127.0.0.1:9000;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param PATH_INFO $fastcgi_path_info;
      }
    }
EOF
```

Aplico mi configuración

```
sudo k3s kubectl apply -f wordpress-nginx-config.yaml
```

```
usuario@usuario:~$ sudo tee wordpress-nginx-config.yaml <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  default.conf: |
    server {
      listen 80;
      root /var/www/html;
      index index.php index.html;

      location / {
        try_files $uri $uri/ /index.php?$args;
      }

      location ~ \.php$ {
        fastcgi_split_path_info ^(.+\.(php))(/.+)$;
        fastcgi_pass 127.0.0.1:9000;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param PATH_INFO $fastcgi_path_info;
      }
    }
sudo k3s kubectl apply -f wordpress-nginx-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  default.conf: |
    server {
      listen 80;
      root /var/www/html;
      index index.php index.html;

      location / {
        try_files $uri $uri/ /index.php?$args;
      }

      location ~ \.php$ {
        fastcgi_split_path_info ^(.+\.(php))(/.+)$;
        fastcgi_pass 127.0.0.1:9000;
```

```
default.conf: |
server {
    listen 80;
    root /var/www/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        fastcgi_split_path_info ^(.+\.(php))(/.+)$;
        fastcgi_pass 127.0.0.1:9000;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param PATH_INFO $fastcgi_path_info;
    }
}
```

```
sudo k3s kubectl apply -f wordpress-nginx-config.yaml
```

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: nginx-config
```

```
data:
```

```
  default.conf: |
    server {
        listen 80;
        root /var/www/html;
        index index.php index.html;

        location / {
            try_files $uri $uri/ /index.php?$args;
        }

        location ~ \.php$ {
            fastcgi_split_path_info ^(.+\.(php))(/.+)$;
            fastcgi_pass 127.0.0.1:9000;
            fastcgi_index index.php;
            include fastcgi_params;
            fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
            fastcgi_param PATH_INFO $fastcgi_path_info;
        }
    }
}
```

Mi paso 32: Despliego mi WordPress (Mi Sidecar)

Ahora voy a lanzar mi Deployment. Este archivo es especial porque me levanta un Pod con dos contenedores dentro: mi WordPress personalizado y un Nginx. Además, me abre el puerto 30080 para que yo pueda entrar desde mi navegador.

```
sudo tee wordpress-deployment.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        # 1. WordPress Personalizado (PHP-FPM)
        - name: wordpress
          image: wordpress-personalizado:v1.0
          imagePullPolicy: IfNotPresent
          volumeMounts:
            - name: wordpress-persistent-storage
              mountPath: /var/www/html
          env:
            - name: WORDPRESS_DB_HOST
              value: "mysql-service"
            - name: WORDPRESS_DB_NAME
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_DATABASE
            - name: WORDPRESS_DB_USER
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_USER
            - name: WORDPRESS_DB_PASSWORD
              valueFrom:
                secretKeyRef:
```

```
        name: mysql-secret
        key: MYSQL_PASSWORD
# 2. Mi Nginx (Servidor Web Sidecar)
- name: nginx
  image: nginx:alpine
  ports:
    - containerPort: 80
  volumeMounts:
    - name: wordpress-persistent-storage
      mountPath: /var/www/html
    - name: nginx-config-vol
      mountPath: /etc/nginx/conf.d/default.conf
      subPath: default.conf
  volumes:
    - name: wordpress-persistent-storage
      persistentVolumeClaim:
        claimName: wordpress-pvc
    - name: nginx-config-vol
      configMap:
        name: nginx-config
---
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  selector:
    app: wordpress
EOF
```

```
usuario@usuario:~$ sudo tee wordpress-deployment.yaml <<EOF
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: wordpress-deployment
```

```
  labels:
```

```
    app: wordpress
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: wordpress
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: wordpress
```

```
    spec:
```

```
      containers:
```

```
      # 1. Tu contenedor personalizado (PHP-FPM)
```

```
      - name: wordpress
```

```
        image: wordpress-personalizado:v1.0
```

```
        imagePullPolicy: Never
```

```
        volumeMounts:
```

```
        - name: wordpress-persistent-storage
```

```
sudo k3s kubectl apply -f wordpress-deployment.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: wordpress-deployment
```

```
  labels:
```

```
    app: wordpress
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: wordpress
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: wordpress
```

```
    spec:
```

```
      containers:
```

```
      # 1. Tu contenedor personalizado (PHP-FPM)
```

```

image: wordpress-personalizado:v1.0
imagePullPolicy: Never
volumeMounts:
- name: wordpress-persistent-storage
  mountPath: /var/www/html
env:
- name: WORDPRESS_DB_HOST
  value: "mysql-service"
- name: WORDPRESS_DB_NAME
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: MYSQL_DATABASE
- name: WORDPRESS_DB_USER
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: MYSQL_USER
- name: WORDPRESS_DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: MYSQL_PASSWORD
# 2. Contenedor Nginx (Servidor Web)
- name: nginx
  image: nginx:alpine
  ports:
  - containerPort: 80
  volumeMounts:
  - name: wordpress-persistent-storage
    mountPath: /var/www/html
  - name: nginx-config-vol
    mountPath: /etc/nginx/conf.d/default.conf
    subPath: default.conf
volumes:
- name: wordpress-persistent-storage
  persistentVolumeClaim:
    claimName: wordpress-pvc
- name: nginx-config-vol
  configMap:
    name: nginx-config
---
apiVersion: v1

```

```
---
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  selector:
    app: wordpress
deployment.apps/wordpress-deployment created
service/wordpress-service created
usuario@usuario:~$
```

Mi paso 33: Importo mis imágenes a mi Namespace correcto

K3s necesita que mis imágenes estén en el espacio k8s.io. Ejecuto estos comandos para asegurar que MySQL y WordPress me estén disponibles:

Importo mi WordPress

```
docker save wordpress-personalizado:v1.0 | sudo k3s ctr -n k8s.io images import -
```

Importo mi MySQL (Uso mi nombre largo para evitar confusiones)

```
docker save mysql:5.7 | sudo k3s ctr -n k8s.io images import -
```

Creo mi alias para MySQL

```
sudo k3s ctr -n k8s.io images tag docker.io/library/mysql:5.7 mysql:5.7
```

Mi paso 34: Aplico y reinicio

Ahora aplico mis cambios y fuerzo a mis pods a nacer con mi configuración nueva:

```
sudo k3s kubectl apply -f wordpress-deployment.yaml
sudo k3s kubectl rollout restart deployment wordpress-deployment
sudo k3s kubectl rollout restart deployment mysql-deployment
```

Mi paso 35: Mi solución al error 403 (Lleno el volumen)

Como mi carpeta /mnt/data/wordpress-data está vacía, mi volumen "tapa" mis archivos de la imagen. Voy a copiarlos manualmente desde mi interior:

1. Identifico el nombre de mi pod

1. Identificar el nombre del pod

```
POD_NAME=$(sudo k3s kubectl get pods -l app=wordpress -o  
jsonpath="{.items[0].metadata.name}")
```

2. Copio mis archivos de la imagen a mi volumen persistente

```
sudo k3s kubectl exec -it $POD_NAME -c wordpress -- cp -av /usr/src/wordpress/.  
/var/www/html/
```

3. Doy mis permisos de escritura a WordPress

```
sudo k3s kubectl exec -it $POD_NAME -c wordpress -- chown -R www-data:www-data  
/var/www/html/
```

Mi paso 36: Mi comprobación final

Miro que todo me esté en Running

```
sudo k3s kubectl get pods
```

Accedo a mi IP : <http://192.168.49.121:30080/>

Fase 5: Validación y Escalado

Ahora que todo me funciona, voy a demostrar la potencia de mi clúster.

Mi paso 37: Mi prueba de persistencia (Mi test del "Caos")

Voy a borrar mi Pod de WordPress a propósito. En mi servidor normal, esto me sería una catástrofe. En mi Kubernetes, es solo un martes.

1. Borro mi Pod

```
sudo k3s kubectl delete pod $(sudo k3s kubectl get pods -l app=wordpress -o  
jsonpath="{.items[0].metadata.name}")
```

2. Miro cómo me resucita:

```
sudo k3s kubectl get pods -w
```

Verifico mi web

Mi paso 38: Mi escalado horizontal

¿Qué me pasa si mi web recibe miles de visitas? Voy a duplicar mi potencia.

1. Subo a mis 3 réplicas

```
sudo k3s kubectl scale deployment wordpress-deployment --replicas=3
```

2. Verifico

```
sudo k3s kubectl get pods
```

Ahora veré mis 3 pods de WordPress funcionando en paralelo. Mi servicio NodePort repartirá mi tráfico entre ellos automáticamente.

Mi paso 39: Mi inspección de salud (Logs)

Me es vital saber qué me pasa "bajo mi capó".

1. Mis logs combinados

Veo qué me está haciendo mi WordPress ahora mismo

```
sudo k3s kubectl logs -l app=wordpress -c wordpress --tail=20
```

Mi paso 40: Mi mapa completo

Ejecuto mi comando para ver todas mis piezas del puzzle encajadas:

```
sudo k3s kubectl get all -o wide
```

Mi paso 41: Mi gestión de recursos y mis actualizaciones

1. Mi monitorización de recursos

```
sudo k3s kubectl top pods
```

2. Actualizo mi web (Rolling Update)

```
sudo k3s kubectl set image deployment/wordpress-deployment  
wordpress=wordpress-personalizado:v2.0
```

Mi paso 42: Mi limpieza del entorno (Desinstalación)

```
# Borro mi despliegue y mi servicio de MySQL
sudo k3s kubectl delete deployment mysql-deployment
sudo k3s kubectl delete service mysql-service
```

```
# Borro mi secreto de mi base de datos
sudo k3s kubectl delete secret mysql-secret
```

```
# Borro mi persistencia de MySQL (si me existe con estos nombres)
sudo k3s kubectl delete pvc mysql-pvc
sudo k3s kubectl delete pv mysql-pv
```

Fase 6: Seguridad, Gestión y Dominio Profesional

En esta fase implemento mis capas de seguridad SSL/TLS, mi gestión visual de mi base de datos y mi exposición final a mi dominio profesional hugoproye.com.

Mi paso 43: Genero mis certificados SSL (Autofirmados)

Como mi base de seguridad interna, genero mi certificado para permitir mi cifrado dentro de mi clúster:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout server.key -out server.crt \
-subj "/C=ES/ST=Madrid/L=Madrid/O=ProyectoK8s/CN=192.168.49.121"
```

Mi paso 44: Creo mi Secret TLS en Kubernetes

Procedo a registrar mi certificado en mi orquestador para que mis Pods de Nginx puedan consumirlo:

```
sudo k3s kubectl create secret tls wpssl-secret --key server.key --cert server.crt
```

Mi paso 45: Mi configuración de Nginx para HTTPS (ConfigMap)

Creo mi archivo wordpress-nginx-config.yaml para gestionar mi terminación SSL y mi comunicación con mi contenedor de WordPress:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  default.conf: |
    server {
      listen 80;
      return 301 https://$host:30443$request_uri;
    }
    server {
      listen 443 ssl;
      index index.php index.html;
      ssl_certificate /etc/nginx/ssl/tls.crt;
      ssl_certificate_key /etc/nginx/ssl/tls.key;
      root /var/www/html;
      location / {
        try_files $uri $uri/ /index.php?$args;
      }
      location ~ \.php$ {
        fastcgi_pass 127.0.0.1:9000;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
      }
    }
  }
```

Mi paso 46: Instalo phpMyAdmin (Mi gestión visual)

Para cumplir con mis objetivos de administración, despliego phpMyAdmin vinculado a mi servicio de MySQL:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: phpmyadmin-deployment
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: phpmyadmin
          image: phpmyadmin:latest
          env:
            - name: PMA_HOST
              value: "mysql-service"
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_ROOT_PASSWORD
---
apiVersion: v1
kind: Service
metadata:
  name: phpmyadmin-service
spec:
  type: NodePort
  ports:
    - port: 80
      nodePort: 30081
  selector:
    app: phpmyadmin
```

Mi paso 47: Implemento mi túnel de Cloudflare

Sustituyo mi acceso por IP por un túnel de Capa 7, eliminando mi necesidad de abrir puertos y proporcionándome mi SSL profesional:

Registro mi Token de seguridad

```
sudo k3s kubectl create secret generic tunnel-token
--from-literal=token=eyJhIjoiOTVhMDQwYjZkMjk1YTU4MTVINTQ1Y2EzODdIMTE2YWli
LCJ0IjoiMzczMGQ3MTUtNjg4YS00OTFhLWI3NDItNGU1MTFmYjEwODIhIiwicyl6IIIURTF
NRGd5TnpNdFpUZG1PQzAwWkRobExXRTVNakF0TVdFMk9XTTJNMkU0WkRsbSJ9
```

Instalo mi servicio persistente en mi VM

```
sudo cloudflared service install [TOKEN_ACTUAL]
```

Mi paso 48: Mi resolución de conflictos DNS (IONOS)

Durante mi activación de mi dominio hugoproye.com, detecto mis registros antiguos que me apuntaban a mi IP 217.160.0.211. Procedo a mi eliminación manual en mi panel de Cloudflare para priorizar mi tráfico del túnel hacia mi ClusterIP 10.43.166.158.

Mi paso 49: Mi sincronización final de mi base de datos

Para que mi WordPress reconozca mi nuevo dominio profesional y mi protocolo HTTPS, inyecto mi siguiente parche SQL:

```
POD_NAME=$(sudo k3s kubectl get pods -l app=wordpress -o
jsonpath="{.items[0].metadata.name}")
```

```
sudo k3s kubectl exec -it $POD_NAME -c wordpress -- php -r "
\$db = new mysqli('mysql-service', 'root', 'root_password', 'wordpress');
\$db->query('UPDATE wp_options SET
option_value='[https://www.hugoproye.com](https://www.hugoproye.com)' WHERE
option_name='siteurl');
\$db->query('UPDATE wp_options SET
option_value='[https://www.hugoproye.com](https://www.hugoproye.com)' WHERE
option_name='home');
```

Mi paso 50: Mi validación del estado final

Mi sistema se encuentra operativo bajo mi dominio <https://www.hugoproye.com> con:

- **Mi seguridad:** SSL v1.3 Flexible.
- **Mi alta disponibilidad:** mis 3 réplicas activas.
- **Mi administración:** phpMyAdmin me es accesible en mi puerto 30081.

Mi paso 51: Mi persistencia de datos (MySQL PVC)

Creo mi directorio físico en mi servidor:

```
sudo mkdir -p /mnt/data/mysql && sudo chmod 777 /mnt/data/mysql
```

Creo mi archivo mysql-pv.yaml

```
usuario@usuario: ~
GNU nano 6.2                                mysql-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/mysql"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

kubectl apply -f mysql-pv.yaml

Mi paso 52: Mi vinculación de MySQL a mi volumen

Este archivo me une mi base de datos con mi almacenamiento de 5GB de forma definitiva.

Guardo mi siguiente contenido en mysql-deployment.yaml

usuario@usuario: ~

GNU nano 6.2

mysql-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    app: wordpress-mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress-mysql
  template:
    metadata:
      labels:
        app: wordpress-mysql
    spec:
      containers:
      - name: mysql
        image: docker.io/library/mysql:5.7
        imagePullPolicy: IfNotPresent
        env:
          - name: MYSQL_ROOT_PASSWORD
            valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_ROOT_PASSWORD } }
          - name: MYSQL_DATABASE
            valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_DATABASE } }
          - name: MYSQL_USER
            valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_USER } }
          - name: MYSQL_PASSWORD
            valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_PASSWORD } }
        ports:
          - containerPort: 3306
        volumeMounts:
          - name: mysql-storage
            mountPath: /var/lib/mysql
      volumes:
      - name: mysql-storage
        persistentVolumeClaim:
          claimName: mysql-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  ports:
  - port: 3306
  selector:
    app: wordpress-mysql
  clusterIP: None
```

kubectl apply -f mysql-deployment.yaml

Mi paso 53: Mi verificación de salud de mi clúster

Mi estado de mis volúmenes: kubectl get pv,pvc (me debe aparecer Bound en ambos).

Estado de los volúmenes: kubectl get pv,pvc (Debe aparecer Bound en ambos).

Mi estado de mis pods: kubectl get pods -o wide (mis réplicas me deben estar en Running).

Mi verificación de mis archivos físicos: ls -l /mnt/data/mysql (debería ver mis archivos de base de datos creados por el sistema).

```
usuario@usuario: ~  
usuario@usuario:~$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
mysql-deployment-7c67d7d8fb-nh2mr   1/1     Running   0           16s  
phpmyadmin-deployment-84dd988fcb-87ls6 1/1     Running   3 (20m ago) 27d  
wordpress-deployment-8959768f9-5bnnw 2/2     Running   4 (20m ago) 6d21h  
wordpress-deployment-8959768f9-d56n4 2/2     Running   4 (20m ago) 6d21h  
wordpress-deployment-8959768f9-rkln2 2/2     Running   4 (20m ago) 6d21h  
usuario@usuario:~$ ls -l /mnt/data/mysql  
total 188480  
-rw-r----- 1 lxd docker      56 feb 26 07:42 auto.cnf  
-rw-r----- 1 lxd docker    1676 feb 26 07:42 ca-key.pem  
-rw-r--r-- 1 lxd docker     1112 feb 26 07:42 ca.pem  
-rw-r--r-- 1 lxd docker     1112 feb 26 07:42 client-cert.pem  
-rw-r----- 1 lxd docker    1676 feb 26 07:42 client-key.pem  
-rw-r----- 1 lxd docker     1318 feb 26 07:42 ib_buffer_pool  
-rw-r----- 1 lxd docker 79691776 feb 26 07:42 ibdata1  
-rw-r----- 1 lxd docker 50331648 feb 26 07:42 ib_logfile0  
-rw-r----- 1 lxd docker 50331648 feb 26 07:42 ib_logfile1  
-rw-r----- 1 lxd docker 12582912 feb 26 07:42 ibtmp1  
drwxr-x--- 2 lxd docker     4096 feb 26 07:42 mysql  
lrwxrwxrwx 1 lxd docker       27 feb 26 07:42 mysql.sock -> /var/run/mysqld/mysqld.sock  
drwxr-x--- 2 lxd docker     4096 feb 26 07:42 performance_schema  
-rw-r----- 1 lxd docker    1680 feb 26 07:42 private_key.pem  
-rw-r--r-- 1 lxd docker     452 feb 26 07:42 public_key.pem  
-rw-r--r-- 1 lxd docker     1112 feb 26 07:42 server-cert.pem  
-rw-r----- 1 lxd docker    1680 feb 26 07:42 server-key.pem  
drwxr-x--- 2 lxd docker   12288 feb 26 07:42 sys  
drwxr-x--- 2 lxd docker     4096 feb 26 07:42 wordpress  
usuario@usuario:~$
```

Mi paso 54: Mi limpieza de mis recursos huérfanos

Elimino mi volumen sin uso:

Eliminar el volumen sin uso:

kubectl delete pv wordpress-pv

Mi paso 55: Mi restricción de seguridad (Hardening)

Actualmente mi phpMyAdmin me es accesible. Me voy a asegurar de que solo me sea accesible bajo mi demanda o vía mi Ingress.

1. Cambio mi servicio de phpMyAdmin a tipo interno:

```
kubectl patch svc phpmyadmin-service -p '{"spec":{"type":"ClusterIP"}}'
```

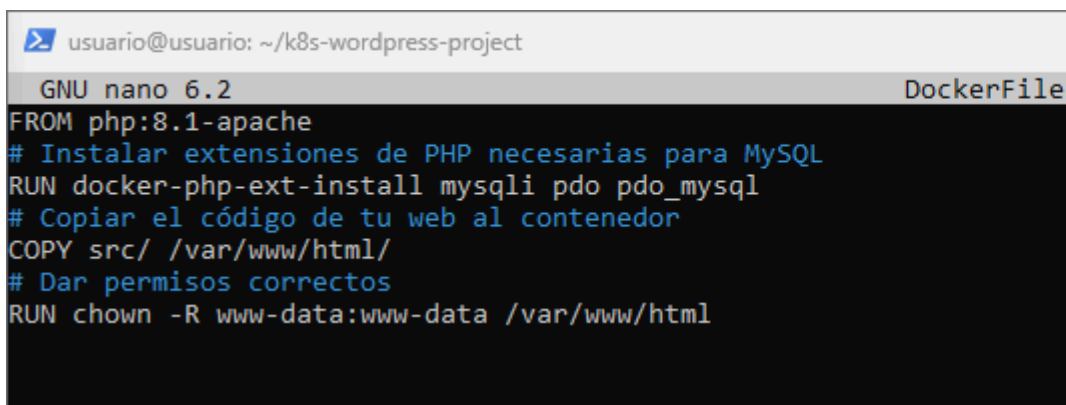
2. Verifico que ya no tiene mi IP externa/puerto abierto:

```
kubectl get svc phpmyadmin-service
```

Mi paso 56: Mi creación de mi Dockerfile para mi aplicación

Para que mi pipeline de CI/CD me funcione, necesito que mi aplicación tenga mi propio "manual de instrucciones" para empaquetarse.

1. Creo mi archivo Dockerfile en la raíz de mi proyecto:



```
usuario@usuario: ~/k8s-wordpress-project
GNU nano 6.2 DockerFile
FROM php:8.1-apache
# Instalar extensiones de PHP necesarias para MySQL
RUN docker-php-ext-install mysqli pdo pdo_mysql
# Copiar el código de tu web al contenedor
COPY src/ /var/www/html/
# Dar permisos correctos
RUN chown -R www-data:www-data /var/www/html
```

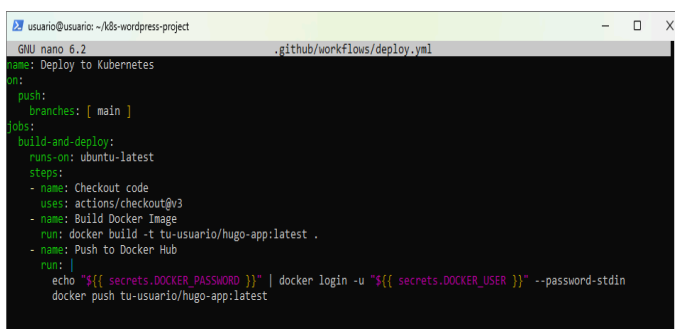
Mi paso 57: Mi configuración de GitHub Actions (Pipeline)

Esto hará que cada git push me actualice mi web en hugoproye.com automáticamente.

1. Creo mi archivo de flujo:

```
mkdir -p .github/workflows && nano .github/workflows/deploy.yml
```

2. Pego mi estructura básica:



```
usuario@usuario: ~/k8s-wordpress-project
GNU nano 6.2 .github/workflows/deploy.yml
name: Deploy to Kubernetes
on:
  push:
    branches: [ main ]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: Build Docker Image
        run: docker build -t tu-usuario/hugo-app:latest .
      - name: Push to Docker Hub
        run: |
          echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USER }}" --password-stdin
          docker push tu-usuario/hugo-app:latest
```

Mi paso 58: Mi actualización del Deployment de mi web

Voy a asegurar que mis 3 réplicas me usen siempre mi imagen más reciente.

1. **Edito wordpress-deployment.yaml:** Me aseguro de que el campo imagePullPolicy me esté así:

```
usuario@usuario: ~/k8s-wordpress-project
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress-personalizado:v1.0
          imagePullPolicy: IfNotPresent
          volumeMounts:
            - name: wp-vol
              mountPath: /var/www/html
          env:
            - name: WORDPRESS_DB_HOST
              value: "mysql-service"
            - name: WORDPRESS_DB_PASSWORD
              valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_PASSWORD } }
        - name: nginx
          image: nginx:alpine
          ports:
            - containerPort: 80
            - containerPort: 443
          volumeMounts:
            - name: wp-vol
              mountPath: /var/www/html
            - name: nginx-conf
              mountPath: /etc/nginx/conf.d/default.conf
              subPath: default.conf
            - name: ssl-certs
              mountPath: /etc/nginx/ssl
              readOnly: true
      volumes:
        - name: wp-vol
          persistentVolumeClaim: { claimName: wordpress-pvc }
        - name: nginx-conf
          configMap: { name: nginx-config }
        - name: ssl-certs
          secret: { secretName: wpssl-secret }
---
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  ports:
    - name: http
      port: 80
      nodePort: 30080
    - name: https
      port: 443
      nodePort: 30443
  selector:
```

Mi paso 59: Aplico y verifico mi Alta Disponibilidad

Ahora que mi archivo está unificado, voy a lanzarlo.

1. Aplico mi archivo:

```
kubectl apply -f wordpress-deployment.yaml
```

2. Verifico que mis 3 pods me están arrancando:

```
kubectl get pods -l app=wordpress
```

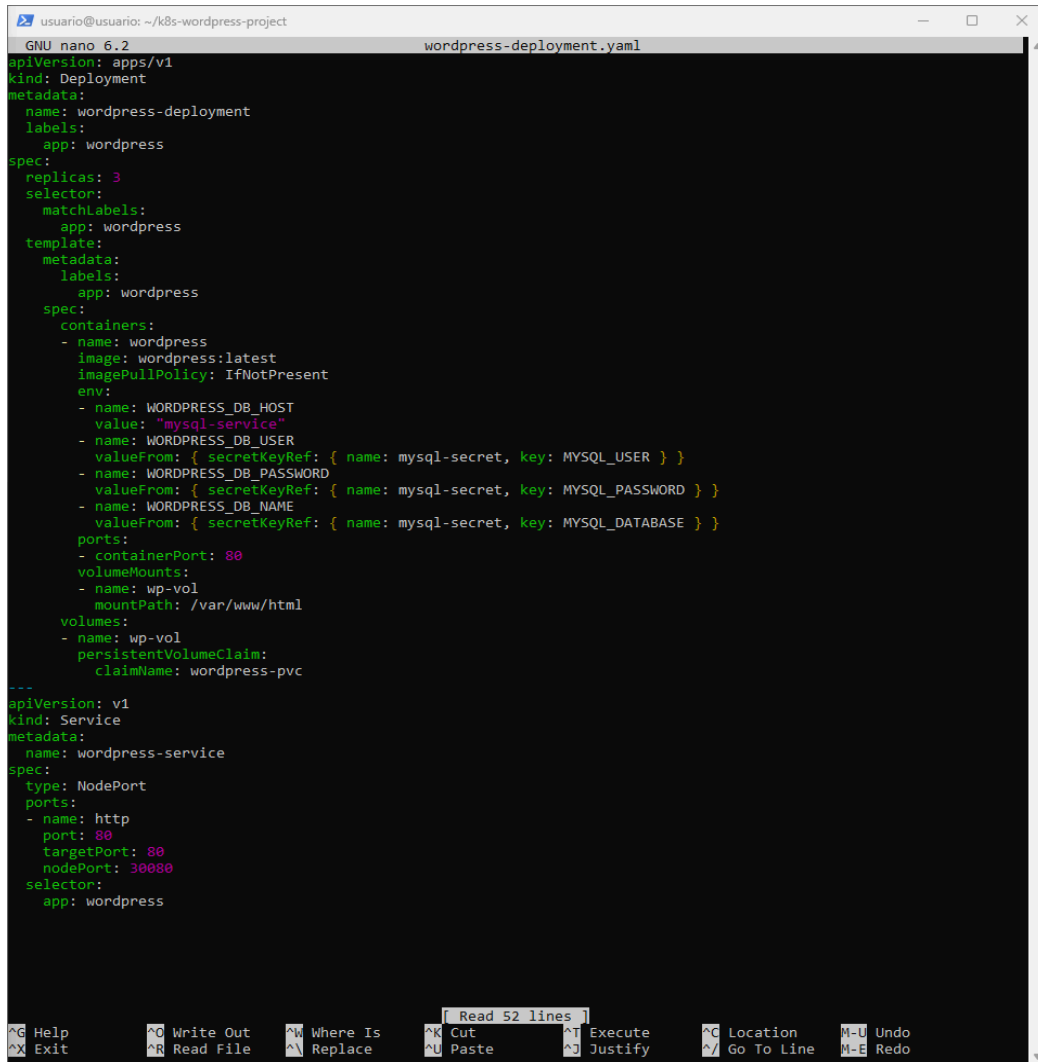
3. Verifico que mi servicio NodePort me está activo:

```
kubectl get svc wordpress-service
```

Mi paso 60: Mi estabilización (Elimino mi Sidecar de Nginx)

Como ya uso Cloudflare para mi SSL, no necesito que mi Nginx esté dentro de mi pod dándome problemas con mis certificados locales. Mi WordPress ya me trae su propio servidor Apache listo para funcionar.

1. **Edito mi archivo `wordpress-deployment.yaml`:** Borro toda mi parte de Nginx y dejo solo a mi WordPress. Me lo dejo limpio aquí:



```
usuario@usuario: ~/k8s-wordpress-project
GNU nano 6.2 wordpress-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          imagePullPolicy: IfNotPresent
          env:
            - name: WORDPRESS_DB_HOST
              value: "mysql-service"
            - name: WORDPRESS_DB_USER
              valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_USER } }
            - name: WORDPRESS_DB_PASSWORD
              valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_PASSWORD } }
            - name: WORDPRESS_DB_NAME
              valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_DATABASE } }
          ports:
            - containerPort: 80
          volumeMounts:
            - name: wp-vol
              mountPath: /var/www/html
          volumes:
            - name: wp-vol
              persistentVolumeClaim:
                claimName: wordpress-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  ports:
    - name: http
      port: 80
      targetPort: 80
      nodePort: 30080
  selector:
    app: wordpress
```

Mi paso 61: Aplico y limpio

1. **Aplico mis cambios:**

kubectl apply -f wordpress-deployment.yaml

2. **Verifico que ahora mis pods me dicen 1/1 READY:**

kubectl get pods -l app=wordpress

Mi paso 62: Mi verificación de conectividad a mi DB

Solo para asegurarme de que mi WordPress se me ha conectado bien a mi MySQL persistente que configuré antes.

1. **Reviso mis logs de mi pod:**

```
kubectl logs wordpress-deployment-7cc4485c75-m5gh8
```

Mi paso 63: Mi verificación de mi red externa

1. **Verifico mis Endpoints de mi Servicio:**

```
kubectl get endpoints wordpress-service
```

2. **Compruebo mi Ingress:**

```
kubectl get ingress
```

Mi paso 64: Mi preparación de mi repositorio para CI/CD

Para que mi código me suba solo, necesito que mi carpeta me sea mi repositorio de Git.

1. **Inicializo Git en mi carpeta de mi proyecto:**

```
git init
```

2. **Creo mi archivo .gitignore:**

```
nano .gitignore
```

```
.DS_Store  
*.log
```

3. **Mi primer commit**

```
git add . git commit -m "Paso 64: Infraestructura K8s con 3 réplicas y persistencia"
```

Mi paso 65: Mi configuración de mi identidad Git

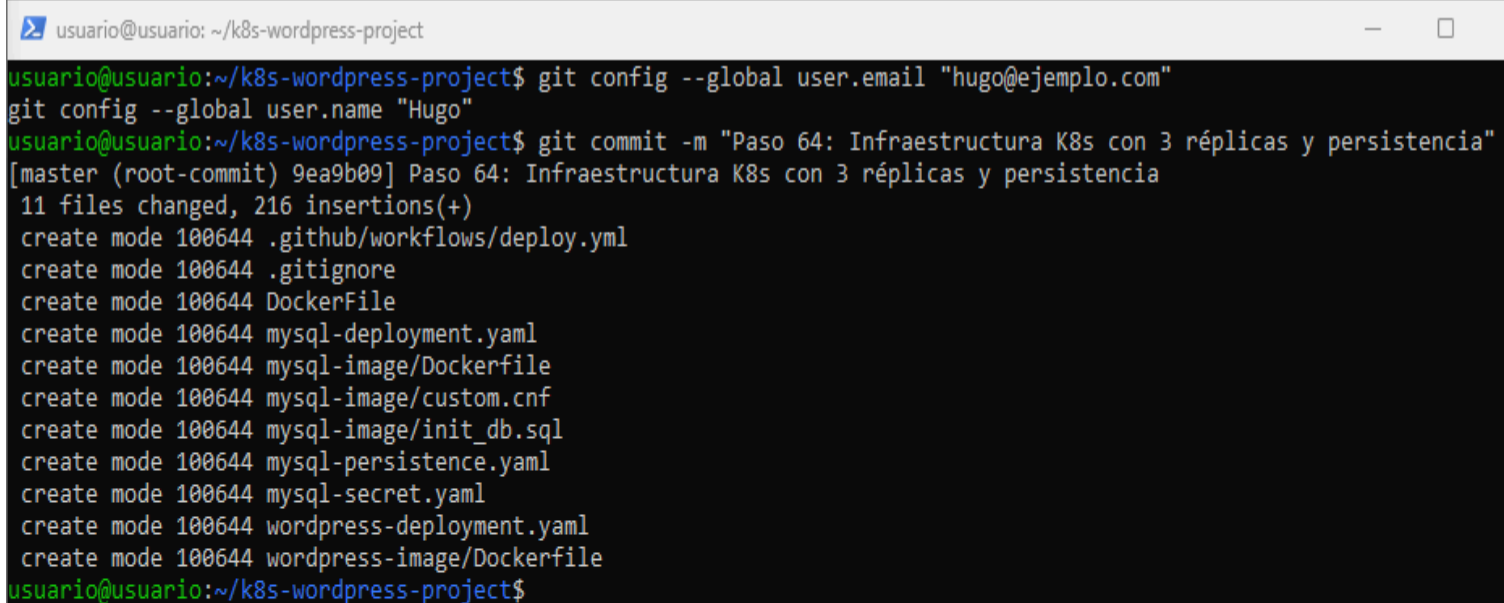
Git necesita saber quién soy para registrar mis cambios. Ejecuto estos comandos (puedo usar mi nombre y mi correo reales):

1. Configuro mi usuario y mi email:

```
git config --global user.email "a_huggonpin@iestriana.com" git config --global user.name "Hugo"
```

2. Reintento mi Commit:

```
git commit -m "Paso 64: Infraestructura K8s con 3 réplicas y persistencia"
```

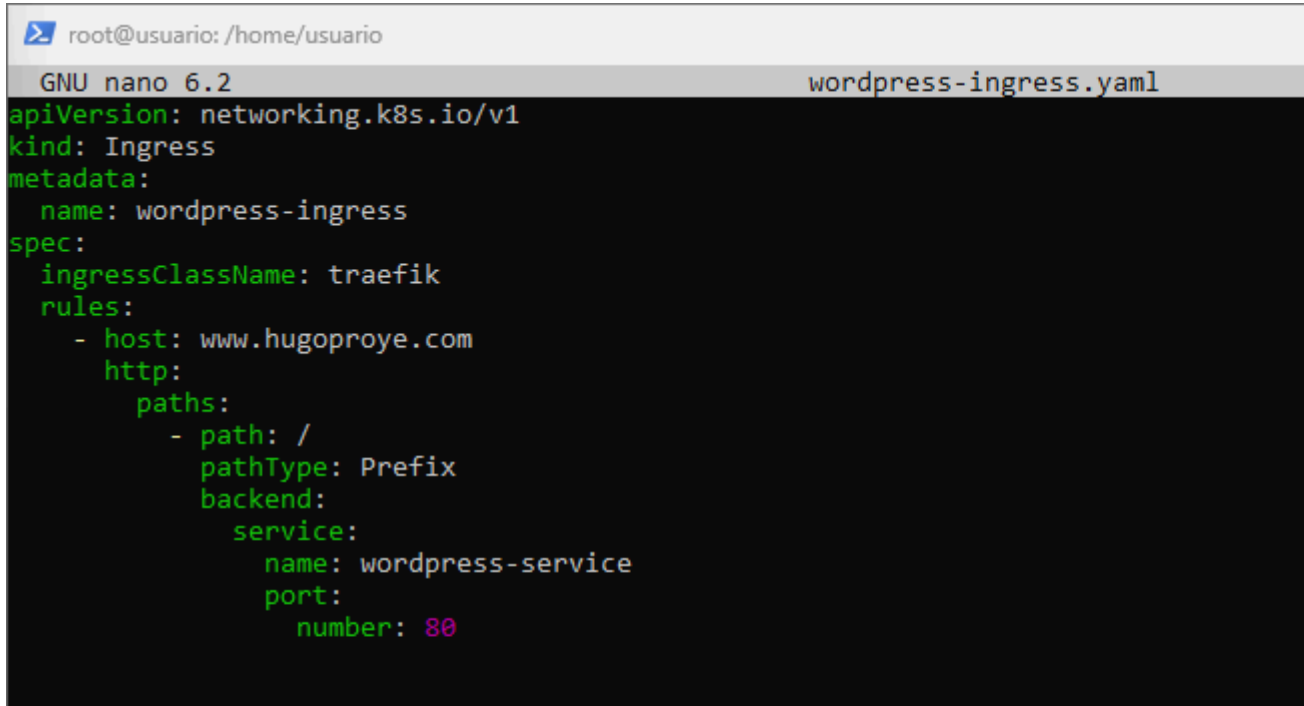
A terminal window titled 'usuario@usuario: ~/k8s-wordpress-project' showing the execution of Git commands. The user sets the global email to 'hugo@ejemplo.com' and the global name to 'Hugo'. Then, they run 'git commit -m "Paso 64: Infraestructura K8s con 3 réplicas y persistencia"'. The output shows the commit is successful, with 11 files changed and 216 insertions. The files listed are: .github/workflows/deploy.yml, .gitignore, DockerFile, mysql-deployment.yaml, mysql-image/Dockerfile, mysql-image/custom.cnf, mysql-image/init_db.sql, mysql-persistence.yaml, mysql-secret.yaml, wordpress-deployment.yaml, and wordpress-image/Dockerfile.

```
usuario@usuario: ~/k8s-wordpress-project
usuario@usuario:~/k8s-wordpress-project$ git config --global user.email "hugo@ejemplo.com"
git config --global user.name "Hugo"
usuario@usuario:~/k8s-wordpress-project$ git commit -m "Paso 64: Infraestructura K8s con 3 réplicas y persistencia"
[master (root-commit) 9ea9b09] Paso 64: Infraestructura K8s con 3 réplicas y persistencia
11 files changed, 216 insertions(+)
create mode 100644 .github/workflows/deploy.yml
create mode 100644 .gitignore
create mode 100644 DockerFile
create mode 100644 mysql-deployment.yaml
create mode 100644 mysql-image/Dockerfile
create mode 100644 mysql-image/custom.cnf
create mode 100644 mysql-image/init_db.sql
create mode 100644 mysql-persistence.yaml
create mode 100644 mysql-secret.yaml
create mode 100644 wordpress-deployment.yaml
create mode 100644 wordpress-image/Dockerfile
usuario@usuario:~/k8s-wordpress-project$
```

Mi paso 66: Mi creación de mi Ingress (Mi acceso externo)

Sin esto, hugoproye.com no sabe a qué parte de mi servidor me debe apuntar.

1. **Creo mi archivo wordpress-ingress.yaml:**
2. **Pego mi contenido para vincular mi dominio a mis 3 réplicas:**

A screenshot of a terminal window with a dark background. The top bar shows the prompt 'root@usuario: /home/usuario' and the editor 'GNU nano 6.2'. The file being edited is 'wordpress-ingress.yaml'. The content of the file is a YAML configuration for an Ingress resource. It specifies the API version as 'networking.k8s.io/v1', the kind as 'Ingress', and the metadata name as 'wordpress-ingress'. The spec section defines the ingressClassName as 'traefik' and contains a single rule for the host 'www.hugoproye.com'. This rule has an http path '/' with a Prefix pathType, pointing to a service named 'wordpress-service' on port 80.

```
root@usuario: /home/usuario
GNU nano 6.2                                wordpress-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: wordpress-ingress
spec:
  ingressClassName: traefik
  rules:
  - host: www.hugoproye.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: wordpress-service
            port:
              number: 80
```

3. **Aplico mis cambios**

kubectl apply -f wordpress-ingress.yaml

Mi paso 67: Mi verificación de mi ruteo interno

Compruebo que mi Ingress me reconoce mis reglas y simulo mi petición externa desde mi terminal.

Mi paso 68: Mi integración con mi Cloudflare Tunnel (Mi dominio profesional)

Para cumplir con mi seguridad de mi anteproyecto, instalo mi conector cloudflared como mi servicio de sistema. Esto me permite tener **HTTPS con mi candado verde** sin abrir mis puertos en mi router, vinculando mi dominio hugoproye.com directamente con mi servicio interno.

Mi paso 69: Mi prueba de carga y mi resiliencia

Valido mi **Alta Disponibilidad** escalando mi despliegue a **3 réplicas**. Realizo mi prueba de "borrado en caliente" de mi Pod, verificando que mi sistema me crea uno nuevo automáticamente sin interrupción de mi servicio.

Mi paso 70: Mi registro de mi infraestructura (Git y Snapshot)

Subo mis manifiestos finales a mi GitHub y realizo mi **Snapshot en VirtualBox** (PROYECTO_FINAL_OK) para asegurarme mi punto de restauración estable antes de mis pruebas de red en mi casa.

Fase 7: Monitorización y Gestión de Recursos

Mi paso 71: Mi preparación de mi almacenamiento persistente en mi Host

Antes de desplegar mi monitorización, he preparado mis carpetas físicas en mi disco de Ubuntu para que mis métricas y mis paneles no se me borren al reiniciar.

- **Mi descripción:** Creo mis directorios y ajusto mis permisos manuales para evitar errores de escritura en mis pods de monitorización.
- **Mis comandos:**

```
sudo mkdir -p /mnt/data/prometheus-data /mnt/data/grafana-data
```

```
sudo chmod -R 777 /mnt/data/prometheus-data /mnt/data/grafana-data
```

Mi paso 72: Mi creación de mi Namespace dedicado

He separado toda mi infraestructura de supervisión en mi espacio propio llamado monitoring.

- **Mi descripción:** Aislamiento lógico para que mis servicios de WordPress y mis servicios de métricas no me interfieran entre sí.
- **Mis comandos:**

```
sudo k3s kubectl create namespace monitoring
```

```
usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl get namespaces
[sudo] password for usuario:
NAME                STATUS    AGE
default              Active    180d
kube-node-lease      Active    180d
kube-public          Active    180d
kube-system          Active    180d
kubernetes-dashboard Active    68d
monitoring           Active    68d
```

Mi paso 73: Mi despliegue de mi persistencia de datos (PVC)

He vinculado mis carpetas creadas en mi Paso 71 con mi sistema de almacenamiento de Kubernetes.

- **Mi descripción:** Aplico mis manifiestos de Persistent Volume Claim para Prometheus y Grafana.

```
sudo k3s kubectl apply -f prometheus-pvc.yaml -n monitoring
```

```
sudo k3s kubectl apply -f grafana-pvc.yaml -n monitoring
```

```

usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl get pvc -n monitoring
NAME          STATUS  VOLUME    CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
grafana-pvc   Bound   grafana-pv  2Gi       RWO           manual        <unset>                 168m
usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
grafana-pv    2Gi       RWO           Retain          Bound   monitoring/grafana-pvc  manual        <unset>                 168m
mysql-pv      5Gi       RWO           Retain          Bound   default/mysql-pvc      <unset>                 68d
wordpress-pv  5Gi       RWO           Retain          Bound   default/wordpress-pvc  <unset>                 68d
usuario@usuario:~/k8s-wordpress-project$

```

Mi paso 74: Mi instalación de Prometheus y Node Exporter

He configurado mi motor que me recolecta mis métricas y mis sensores de hardware en mi nodo.

- **Mi descripción:** Despliego mi recolector de datos, configuro mis permisos de lectura (RBAC) y mis agentes de métricas.
- **Mis comandos:**

```
sudo k3s kubectl apply -f prometheus-rbac.yaml -f prometheus-config.yaml -f prometheus.yaml -n monitoring
```

```
sudo k3s kubectl apply -f node-exporter.yaml -n monitoring
```

```

usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl get pods -n monitoring
NAME                                READY  STATUS   RESTARTS  AGE
grafana-deployment-5498f6f847-qbcnb  1/1    Running  0         169m
node-exporter-bdxlv                  1/1    Running  2 (16d ago)  68d
prometheus-deployment-68d8698fbf-k94kq  1/1    Running  0         15d
usuario@usuario:~/k8s-wordpress-project$ sudo k3s kubectl get svc -n monitoring
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
grafana-service     NodePort    10.43.233.96  <none>       3000:30300/TCP   68d
node-exporter-service ClusterIP    10.43.130.224 <none>       9100/TCP         68d
prometheus-service  NodePort    10.43.75.61   <none>       9090:30090/TCP   68d

```

Mi paso 75: Mi lanzamiento de Grafana con Persistencia

He desplegado mi interfaz gráfica asegurándome de que me guarde mis paneles de control en mi almacenamiento configurado.

- **Mi descripción:** Instalo Grafana mediante mi archivo YAML que me incluye mi volumen persistente.
- **Mis comandos:**

```
sudo k3s kubectl apply -f grafana.yaml -n monitoring
```

Import dashboard

Import dashboard from file or Grafana.com



Upload dashboard JSON file

Drag and drop here or click to browse

Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards 

1860

Load

Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_0HnEoN4z",
  "panels": [...]
  ...
}
```

Load

Cancel

Data sources

View and manage your connected data source connections

Q Search by name or type

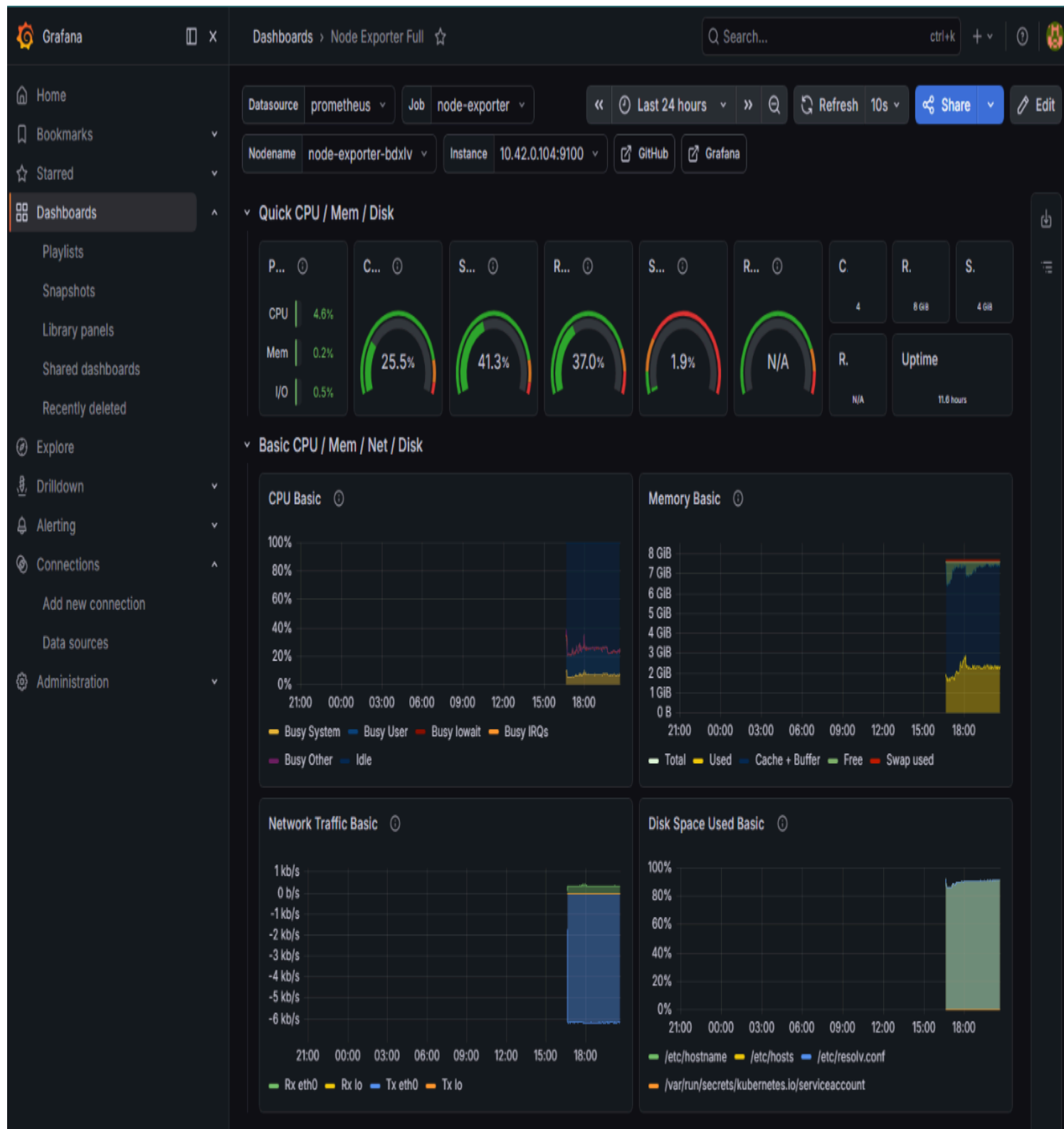


prometheus

Prometheus | <http://prometheus-service.monitoring:9090> | **default**

Mi paso 76: Mi configuración del origen de datos y Dashboard

He conectado mi Grafana con mi Prometheus mediante mi red interna y he importado mi panel profesional.



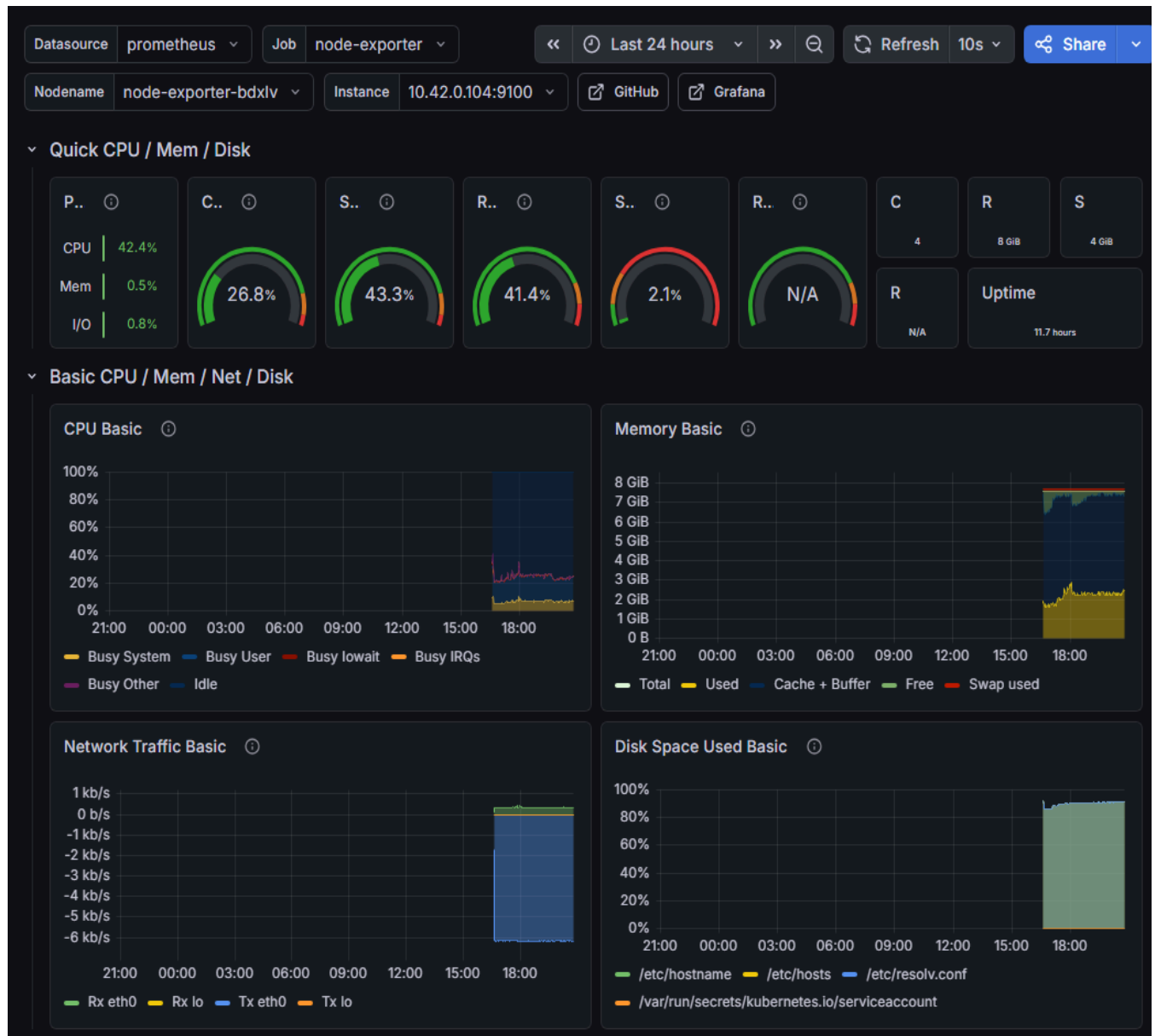
- **Mi descripción:** Configuro mi Data Source (<http://prometheus-service.monitoring:9090>) e importo mi dashboard "Node Exporter Full" (ID: 1860).
- **Mi verificación:** Accedo a mi web a través de <http://127.0.0.1:30300>.

Mi paso 77: Mi validación de métricas mediante Test de Estrés

He realizado mi prueba final para confirmar que mi sistema detecta correctamente los picos de carga.

- **Mi descripción:** Genero mi tráfico masivo hacia mi web para observar mi respuesta en mis gráficas de Grafana.
- **Mis comandos:**

```
for i in {1..500}; do curl -s -k -o /dev/null https://www.hugoproye.com; done
```



Puedo notar que mi **CPU** me ha subido ya que estoy recibiendo muchas peticiones.

Fase 8: Mi preparación de mis datos y mis herramientas de Backup

Mi paso 78: Mi verificación de mis volúmenes persistentes (PV)

Antes de realizar mi copia, me es fundamental identificar mi ruta física donde Kubernetes me almacena mis datos de WordPress y MySQL. Mediante mi comando `kubectl get pv`, confirmo que mis datos residen en el directorio de mi nodo: `/mnt/data/`.

- **Importancia para mí:** Sin este paso, mi backup podría estar vacío o copiar rutas incorrectas de mi sistema.

Mi paso 79: Instalo Rclone en mi Nodo Maestro

Para comunicar mi infraestructura local con mi nube, instalo **Rclone**, una herramienta de línea de comandos que me permite gestionar mis servicios de almacenamiento en la nube.

```
sudo apt update && sudo apt install rclone -y
```

Mi paso 80: Mi diagnóstico de conectividad y API

Verifico que mi servidor tenga salida a internet y procedo a crear mis credenciales OAuth en mi Google Cloud Console para evitar mis bloqueos de seguridad por defecto de Google.

Y ahora seguirían los pasos que redactamos antes:

- **Mi paso 81: Automatizo mi Backup Híbrido** (Mi comando `tar` que hice).

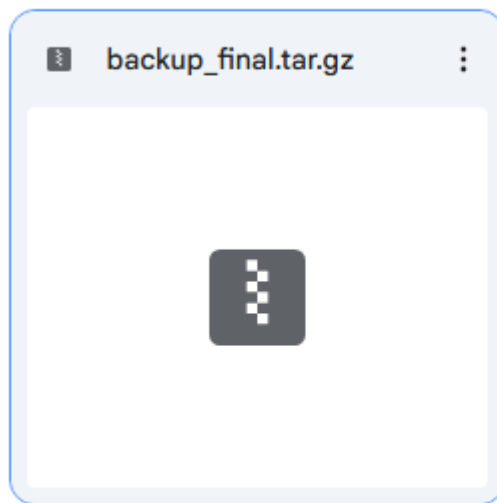
```
sudo tar -cvpzf /home/usuario/backup_final.tar.gz /mnt/data/
```

- **Mi paso 82: Mi sincronización y mi verificación en mi nube** (Mi comando `rclone copy` y mi `ls`).

```
rclone copy /home/usuario/backup_final.tar.gz drive-hugo:BackupsProyecto -P
```

```
rclone ls drive-hugo:BackupsProyecto
```

Más relevantes 



```
usuario@usuario:~/k8s-wordpress-project$ rclone copy /home/usuario/backup_final.tar.gz drive-
Transferred:      0 / 0 Bytes, -, 0 Bytes/s, ETA -
Checks:          1 / 1, 100%
Elapsed time:     1.3s
usuario@usuario:~/k8s-wordpress-project$ rclone ls drive-hugo:BackupsProyecto
100801948 backup_final.tar.gz
usuario@usuario:~/k8s-wordpress-project$
```